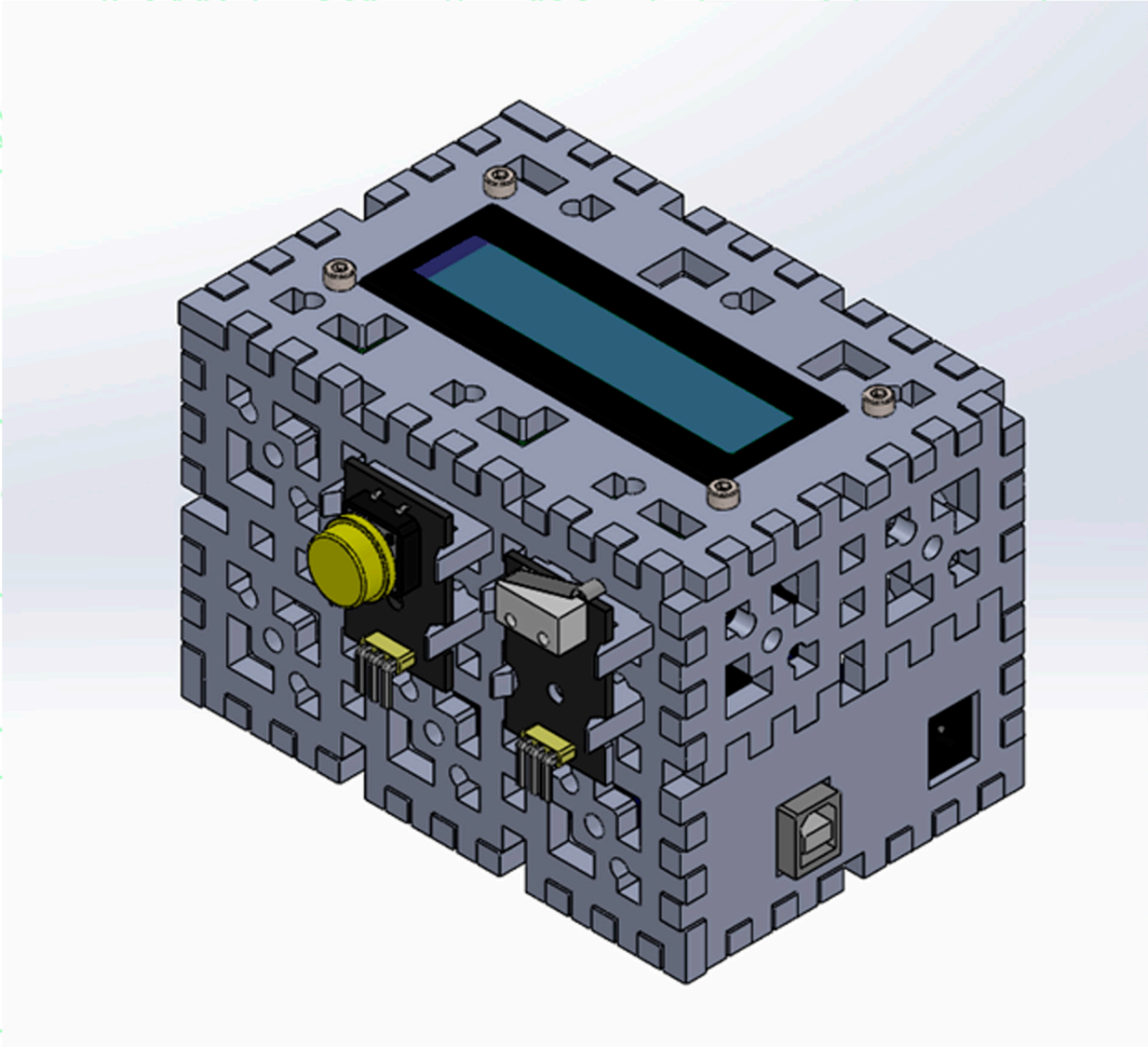
ПРОЕКТ №2
ПЕРЕДАВАЧ МОРЗЕ



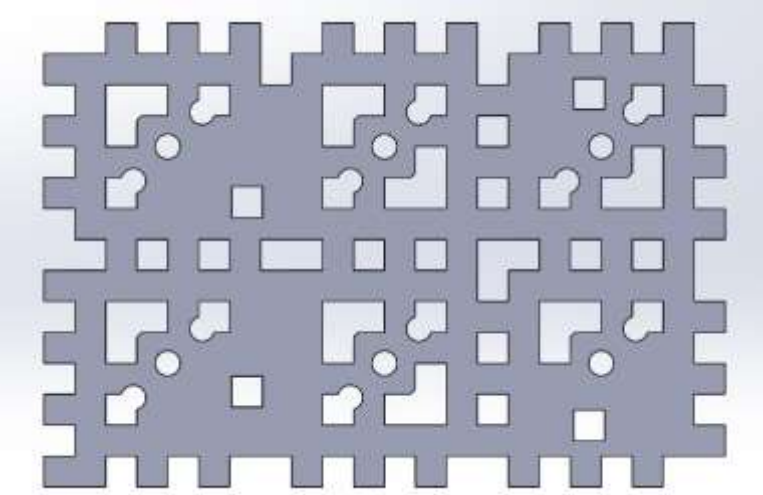
Мета:

- вивчити основні поняття про мікроконтролери;
- покращити навички складання електричних схем;
- покращити навички програмування мікроконтролерів;
- дізнатись принципи кодування інформації.

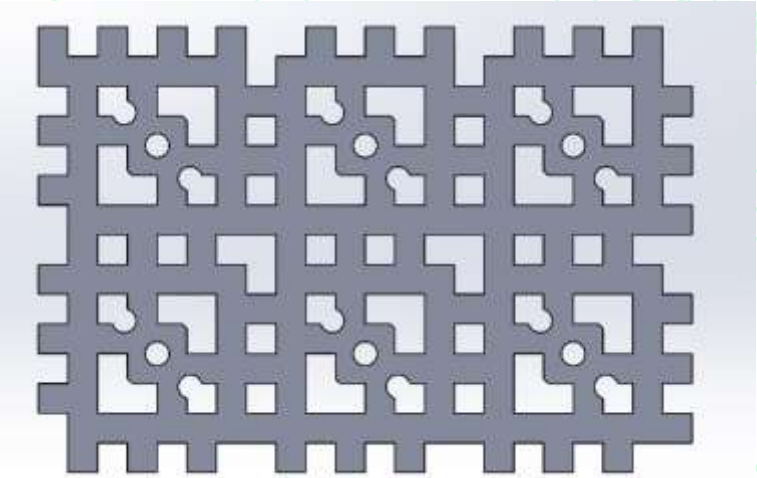
Необхідні електронні компоненти:

- мікроконтролер;
- плата розширення;
- дроти типу F-F, M-F;
- модуль кінцевої кнопки;
- модуль тактової кнопки;
- модуль потенціометра;
- Bluetooth модуль HC-08
- LCD-дисплей;

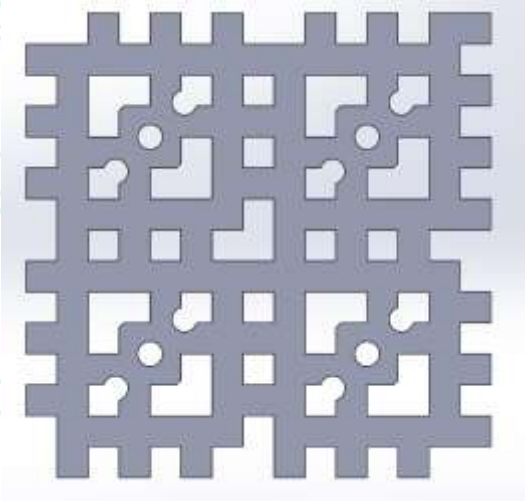
Необхідні конструктивні елементи:



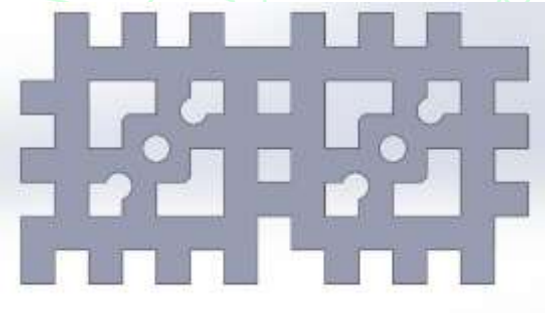
x1. CPlate2x3



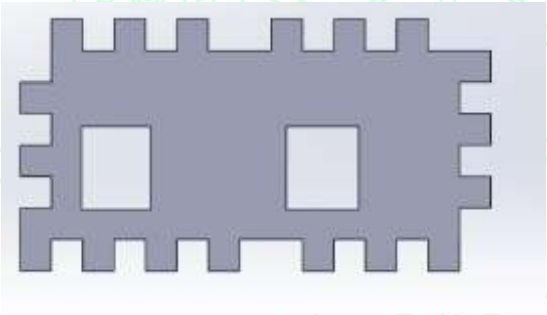
x2. Plate2x3



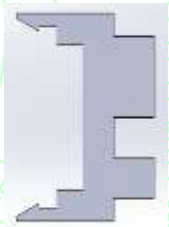
x1. Plate2x2



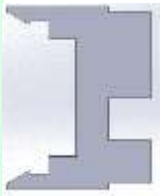
x1. Plate 1x2



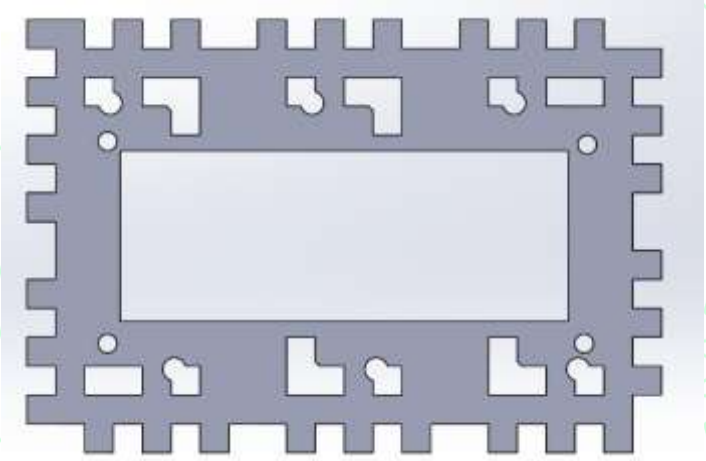
x1. CPlate 1x2



x3. Lfix



x3. Sfix



x1. LCDPlate 1x3



x4. Болт M2.5x10



x4. Гайка M2.5



x4. CPin

Інформація

Азбука Морзе - спосіб знакового кодування, представлення букв алфавіту, цифр, знаків пунктуації та інших символів послідовністю сигналів: довгих (тире) і коротких (точок).

За одиницю часу приймається тривалість однієї точки. Тривалість тире дорівнює трьом точкам. Пауза між елементами одного знака — одна точка, між знаками в слові — 3 точки, між словами — 7 точок.

Була названа на честь американського винахідника Семюеля Морзе, який запропонував її в 1838.

Історія:

Якщо ж говорити про саму телеграфну абетку (систему кодування символів короткими і довгими сигналами для передачі їх по лініях зв'язку, відому як «код Морзе» або «морзянка»), яку застосовують зараз, то вона істотно відрізняється від тієї, що запропонував у 1838 р. С. Морзе, хоча деякі дослідники вважають, що її автором був Альфред Вейл — партнер Самюеля Морзе по бізнесу, відомий тим, що ввів «комерційний код» з груп по 5 символів.

Два інженера - А. Вейл і Д. Генрі - розповіли про європейську розробку - дистанційної мідної котушки, яка здатна передавати електроімпульси. Морзе попросив їх розвивати цю ідею, і в 1837 році на світ з'явився перший телеграфний апарат. Пристрій міг приймати і передавати повідомлення. Пізніше Вейл запропонував систему шифрування за допомогою тире і точок. Таким чином, Морзе не мав прямого відношення до створення азбуки і телеграфу.

За офіційною ж версією, Семюел Морзе був захоплений дивом того часу, а саме отриманням іскри з магнітів. Розгадуючи феномен, він припустив, що за допомогою таких іскор по дротах можна передавати зашифровані повідомлення. Морзе дуже зацікавився цією ідеєю, хоча не мав ні найменшого уявлення навіть про основні принципи роботи електрики. Під час плавання Семюел розробив кілька ідей і накидав деякі креслення своєї задумки. Ще три роки у свого брата на горищі він безуспішно намагався побудувати апарат, який би міг передавати сигнали. При всіх своїх проблемах в пізнанні електрики вивчати її йому було просто ніколи, адже у нього раптово померла дружина, і на ньому залишилися троє маленьких дітей.

Треба зауважити, що вихідна таблиця «коду Морзе» разюче відрізнялася від тих кодів, що сьогодні звучать на аматорських діапазонах. У ній, по-перше, використовувалися посилки трьох різних тривалостей («точка», «тире» і «довге тире» — в 4 рази довше «точки»). По-друге, деякі символи мали паузи усередині своїх кодів.

Принцип кодування азбуки Морзе виходить з того, що літери, які найчастіше вживаються в англійській мові, кодуються простішими сполученнями крапок і тире. Це робить освоєння абетки Морзе простіше, а передачі — компактніше.

Передаватися і прийматися азбука Морзе може з різною швидкістю — це залежить від можливостей і досвіду радистів. Зазвичай середньої кваліфікації радист працює в діапазоні швидкостей 60 — 100 знаків на хвилину. Рекорди за швидкісними прийому-передачі знаходяться в діапазоні швидкостей 220-260 знаків за хвилину.

Ручна передача азбуки Морзе проводиться за допомогою телеграфного ключа (узагальнюючий термін для будь-якого перемикаючого пристрою, використовуваного в основному для передачі знаків абетки Морзе. Подібні ключі використовуються для усіх видів ручної телеграфії, наприклад — в електричному телеграфі і радіотелеграфії) або електронного ключа, також можливо передавати сигнали, різноманітними візуальними засобами, наприклад, за допомогою ліхтаря або сигнальної штори. Прийом коротких

стандартних повідомлень може бути прийнятий без запису, але звичайно весь прийнятий текст повинен бути записаний або вручну, або на друкарській машинці. При прийомі досвідчені радисти роблять запис з відставанням на кілька знаків, що робить прийом спокійнішим і надійним та є показником майстерності того хто приймає.

Азбука Морзе є першим цифровим способом передачі інформації. Телеграф та радіотелеграф спочатку використовували азбуку Морзе; пізніше стали застосовуватися код Бодо та ASCII, які зручніші для автоматизації. Крім того, радіоаматорами розроблена безліч апаратних декодерів азбуки Морзе на базі мікроконтролерів.

Для передачі кирилиці використовувалися коди схожих латинських букв; ця відповідність алфавітів пізніше перейшла в МТК-2, а потім в KOI-7 і KOI-8 (проте в абетці Морзе букві Q відповідає Щ, а в МТК і KOI — Я).

В 2004 Міжнародний союз електрозв'язку (МСЕ) ввів в азбуку Морзе новий код для символу @, для зручності передачі адрес електронної пошти.

На практиці замість візуального заучування комбінацій крапок та тире запам'ятовують так званий «наспів» (мнемонічну словесну форму), що звучанням нагадує звуковий сигнал, яким закодовано певну літеру, цифру чи розділовий знак.

Існує щонайменше два варіанти азбуки Морзе українською мовою, які незначно відрізняються. Стандартом довжини звукових (або світлових) сигналів вважається, коли довгий сигнал (що позначається тире) втричі довший за короткий (що позначався крапкою), пауза між звуковими сигналами одного символу дорівнює тривалості короткого сигналу, пауза між символами дорівнює тривалості довгого сигналу, пауза між словами — тривалості двох довгих сигналів.

Букви			Код Морзе	Наспіви (Пласт)	Наспіви (Регламент)
Латинської абетки	Української абетки (Пласт)	Української абетки (Регламент)			
A	А		• –	А-зот	ай-дааа
B	Б		– • • •	Бон-да-рів-на	бааа-ки-те-чуть
C	Ц		– • – •	«Цьо-ця Дор-ця»	ціїі-ни-ціїі-ни
D	Д		– • •	До-ли-на	дооо-мі-шки
E	Е		•	Ера	ерг
F	Ф		• • – •	Фі-лан-тро-пи	фі-лан-трооо-пи
G	Ґ	Г, Ґ	– – •	Ґо-ґо-дзи	гааа-рааа-жик
H	Г	Х	• • • •	Га-ра-кі-рі	хо-ло-до-чок
I	І	І, Ї	• •	І-ван	І-ди
J	Й		• – – –	Йди хо-лод-ком	йди-хооо-лооод-кооом
K	К		– • –	Ко-лі-но	кооо-ва-ниий
L	Л		• – • •	Лі-со-ру-би	лу-нааа-ти-ки
M	М		– –	Мо-роз	мооо-рзеее
N	Н		– •	Но-та	нооо-мер
O	О		– – –	О-ко-лот	оооо-коооо-лооот

Букви			Код Морзе	Наспіви (Пласт)	Наспіви (Регламент)
Латинської абетки	Української абетки (Пласт)	Української абетки (Регламент)			
P	П		• – – •	При-мо-роз-ки	пи-лааа-пиии-лить
Q	Ш	Щ	– – • –	Шо-ло-ви-ло	щууу-кааа-жи-ваaa
R	Р		• – •	Ри-бонь-ка	ру-кааа-ми
S	С		• • •	Са-ха-ра	са-мо-кат
T	Т		–	Том	тоooон
U	У		• • –	У-па-док	у-ні-сооон
V	Ж		• • • –	Жа-б'є се-ло	я-бук-ва-жеее
W	В		• – –	Ві-тро-гон	ви-сооо-киий
X	Ь		– • • –	То м'я-кий тон	тооо-м'я-кий-знааак
Y	И		– • – –	То не во-но	виии-гляд-диии-вниий
Z	З		– – • •	Зо-ло-ту-ха	зааа-кааа-та-но
Đ, É, Ę	Є		• • – • •	Єд-ність то сила	е-ле-ктрооо-ні-ка
Ó, Ö, Ø	Ч		– – – •	Чор-но-го-ра	чооо-лооо-віii-чок
CH, Ĥ, Š	Х	Ш	– – – –	Хо-ло-до-чок	шаaa-рооо-ваaa-риии
Ü, Ŭ	Ю		• • – –	Ю-рій ко-роль	ю-лі-ааа-наaa
Ä, Æ, Ą	Я		• – • –	Я-ро-шен-ко	я-киий-я-киий
Ĵ	Ї		• – – – •	Їж, хто го-ло-ден	ї-їїї-неее-маaa-є
Ń, Ñ	Щ		– – • – –	Що-ро-ку мо-роз	

Цифри та знаки:

Коди цифр простіше запам'ятати, знаючи, що кожен з них складається з 5 сигналів (крапок чи тире), цифри 0—5 мають стільки ж крапок на початку, як значення цифри, а цифри 6—9 — стільки крапок укінці, скільки не вистачає до десяти, решта сигналів у коді — тире. Регламент аматорського радіозв'язку України також дозволяє у контексті «цифрових текстів» (довгих послідовностей цифр) зливати послідовності тире в нулі і дев'ятки у одне довге тире.

Символ	Код Морзе	Наспіви
1	• – – – –	це-ооо-диии-ниии-цяяя
2	• • – – –	два-на-гііі-ркууу-йшлооо
3	• • • – –	три-то-бі-мааа-лооо
4	• • • • –	чо-ти-ри-ки-таaa
5	• • • • •	п’я-ти-кут-ни-ки
6	– • • • •	шііість-по-ки-що-є
7	– – • • •	даааай-даааай-за-ку-рить
8	– – – • •	вооо-сьмооо-гооо-чис-ла
9	– – – – •	деееe-в’яяя-тиии-сооо-тий
0	– – – – –	нуууууль-сааа-миииий-мааа-лиииий
9 (в цифрових текстах)	———— •	
0 (в цифрових текстах)	————	
Крапка (.)	• • • • • •	кра-по-чка-кра-по-чка
Кома (,)	• – • – • –	крю-чоook-крю-чоook-крю-чоook
Скісна риска (/)	– • • – •	дрооо-бо-ва-риииc-ка
Знак питання (?)	• • – – • •	за-пи-тааан-няяяч-ка-ми
Знак оклику (!)	– – • • – –	пооо-клиииич-ме-не-друуу-жеее
Двокрапка (:)	– – – • • •	двооо-краaa-пкааа-на-дій-шла
Крапка з комою (;)	– • – • – •	краaa-пка-кооо-ма-рааа-зом
Дужки (())	– • – – • –	дууу-жку-меее-ніii-пи-шиии
Апостроф (’)	• – – – – •	крю-чоооо-чоook-звееер-хууу-став
Тире (—), Дефіс (-)	– • • • • –	риии-соч-ку-тут-пи-шиии
Лапки («»)	• – • • – •	ла-пкиии-роз-міс-тиии-лись
Знак розділу	– • • • –	слууу-хай-те-ме-неее
Комерційне АТ(@)	• – – • – •	со-бааа-кааа-гав-куuу-ча
Символ долара ($)	• • • – • • –	
Помилка/перебій	• • • • • • • •	шість-на-ві-сім-со-рок-ві-сім
Кінець зв’язку	• • – • –	до-по-бааа-чен-няяя

Часто для прискорення радіообміну використовуються абревіатури, умовні вирази та спеціальні «Q-коди», наприклад:

- CQ — викликаю всіх;
- OM — друже, приятелю;
- PSE — будь ласка;
- RX — приймач;
- TX — передавач;
- QSL — підтверджую прийом;

- QRZ? — хто мене викликає?;
- SK — кінець зв'язку;
- 55 — дружнє «рукостискання»;
- 73 — найкращі побажання;
- 88 — любов і поцілунок (жартівливо адресується жінкам-радистам);
- 99 — не бажаю з Вами працювати.

Мало хто сьогодні цікавиться такою застарілою технологією як азбука Морзе та телеграф, однак як не дивно навіть сьогодні вони ще використовуються для зв'язку.

Цікаві факти:

1. Хоч азбуку Морзе було винайдено понад 225 років, тому її вивчають навіть сьогодні при підготовці розвідників та шпигунів.

2. Азбуку Морзе також називають «Вікторіанським інтернетом» оскільки свого часу телеграф і азбуку Морзе використовували практично по всьому світі для передачі повідомлень протягом багатьох десятиліть.

3. Мало хто знає, що винахідник азбуки Морзе — Самуель Морзе є також одним з найвидатніших художників-портретистів в США.

4. Конгрес США свого часу виділив понад 30000 доларів на створення першої телеграфної лінії. В ті часи це була неймовірно велика сума.

5. 2007 року японська компанія випустила наручний годинник з азбукою Морзе. Такий незвичний годинник повідомить час, використовуючи азбуку Морзе, після натискання кнопки «play».

6. Є навіть специфічний спорт по швидкості передачі та розпізнавання тексту азбукою Морзе — швидкісна телеграфія.

Переваги:

- Висока перешкодозахищеність при прийомі на слух в умовах сильних радіоперешкод;
- Можливість кодування вручну;
- Запис і відтворення сигналів найпростішими пристроями.

Недоліки:

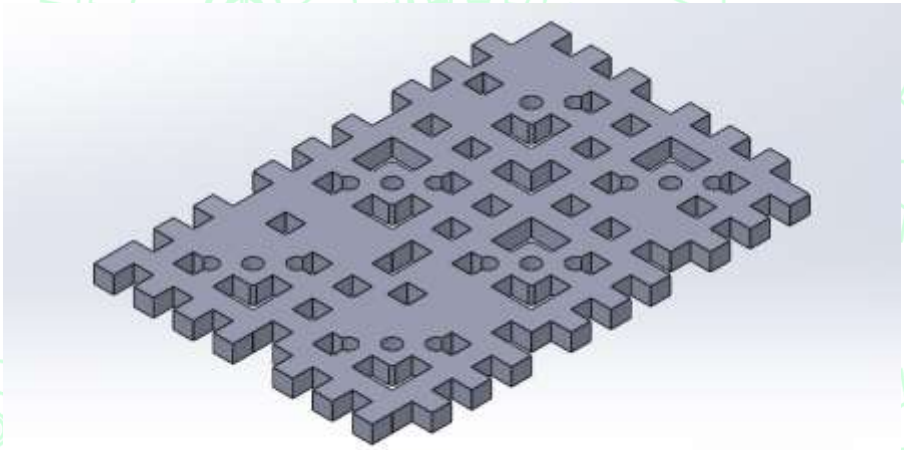
- Неекономічність, на передачу одного знаку коду потрібно в середньому 9,5 елементарних посилок;
- Мала придатність для букводрукуючого прийому;
- Низька швидкість телеграфування.

Сигнал Звук SOS забороняється подавати, якщо немає неминучої загрози для життя людей або судна на морі. SOS подається без пауз між буквами: «· · · - - - · · ·» (три крапки, три тире, три крапки), тобто як одна довга буква. Хоча часто вважається, що SOS є абревіатурою від «Save our souls» (врятуйте наші душі) або «Save our ship» (врятуйте наш корабель), насправді він був обраний через простоту передачі та запам'ятовування, до того ж передається не так як всі абревіатури (окремими літерами), а єдиною літерою.

В цій проектній роботі ви створите свій власний передавач/приймач азбуки Морзе, сигнали будуть оброблятись мікроконтролером, та відправлятись за допомогою протоколу Bluetooth.

Збірка моделі

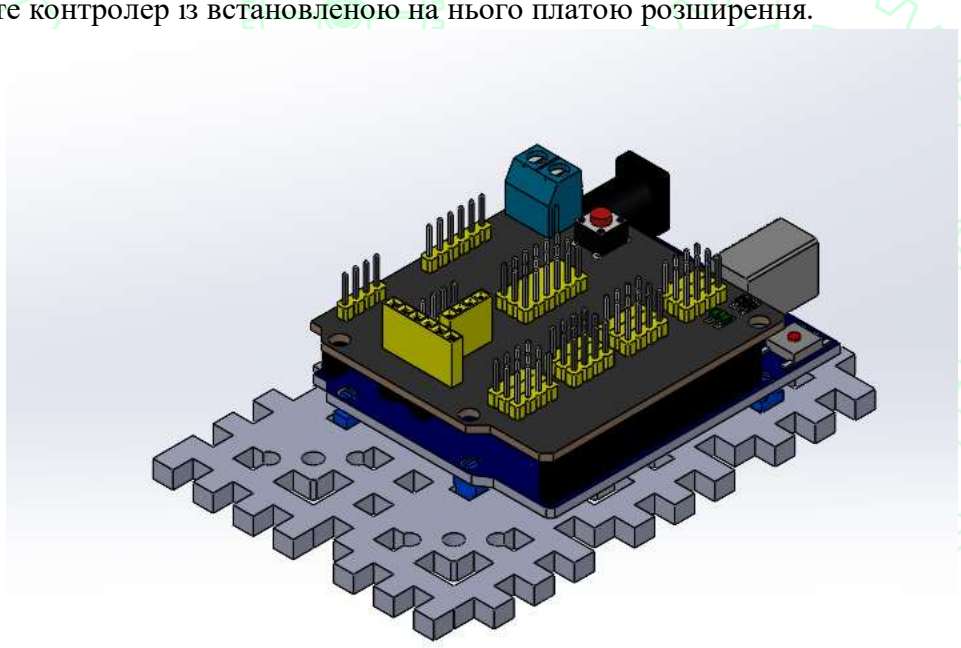
1. Підготуйте необхідні конструктивні елементи. Основою моделі буде пластина для кріплення мікроконтролера.



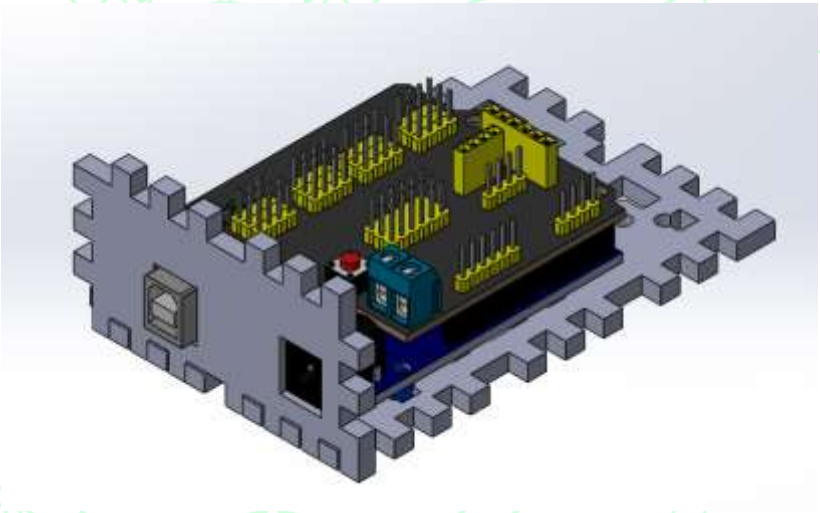
2. Додайте піни для кріплення мікроконтролера до пластини.



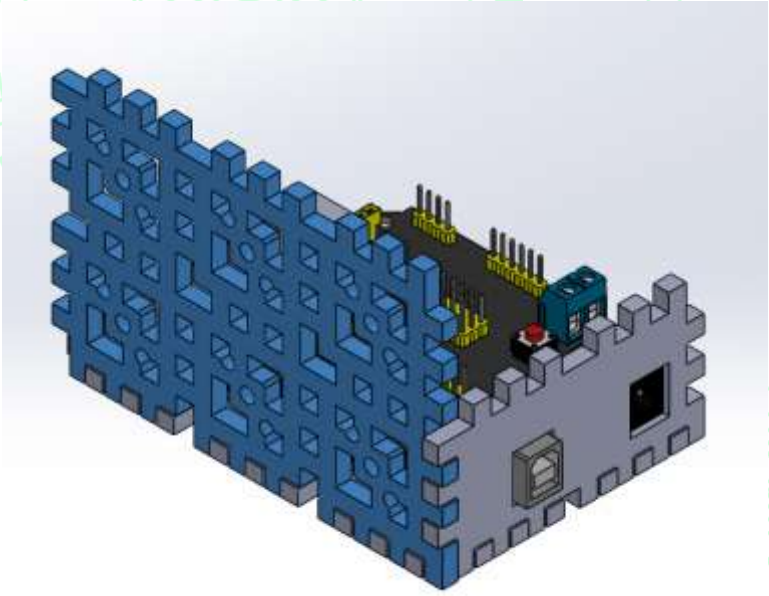
3. Додайте контролер із встановленою на нього платою розширення.



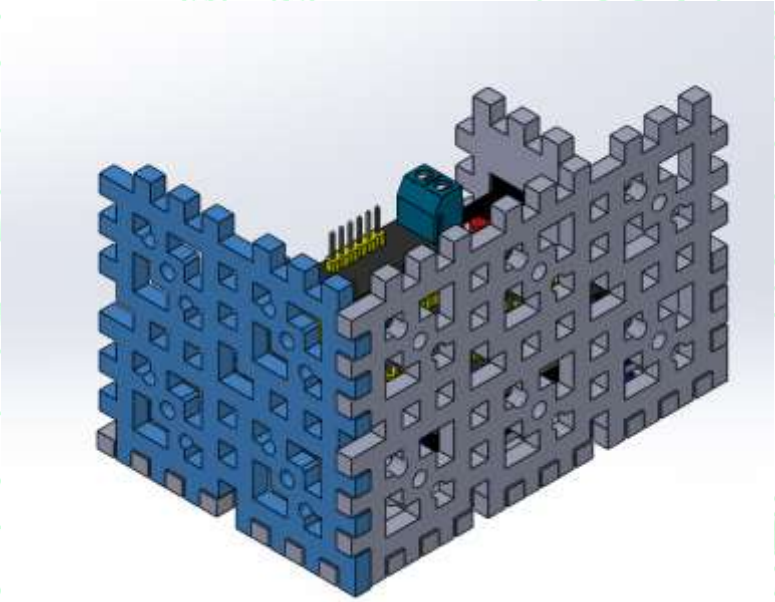
4. Додайте пластину 1x2 з вирізами під мікроконтролер.



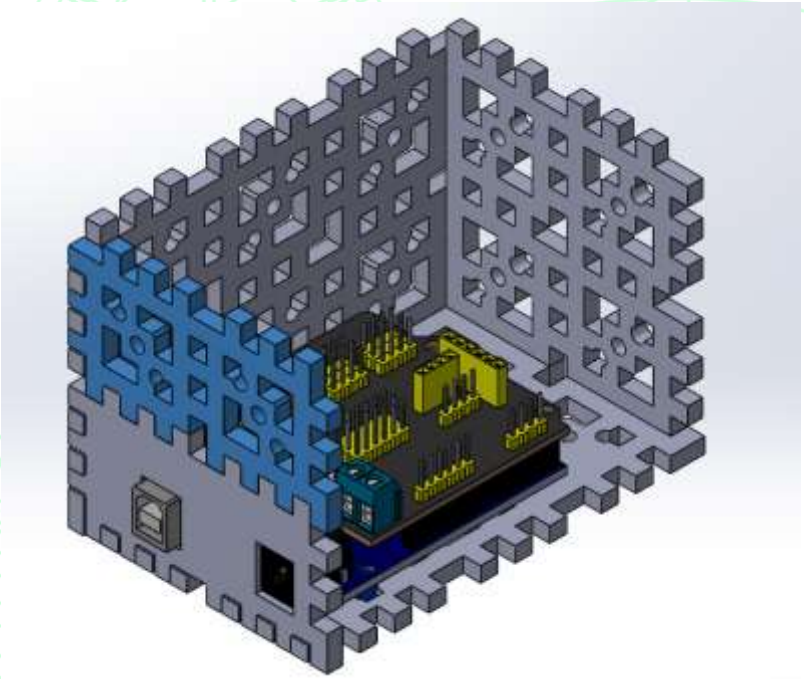
5. Додайте пластину 2x3 та зафіксуйте як показано на рисунку, для формування загальної конструкції корпусу.



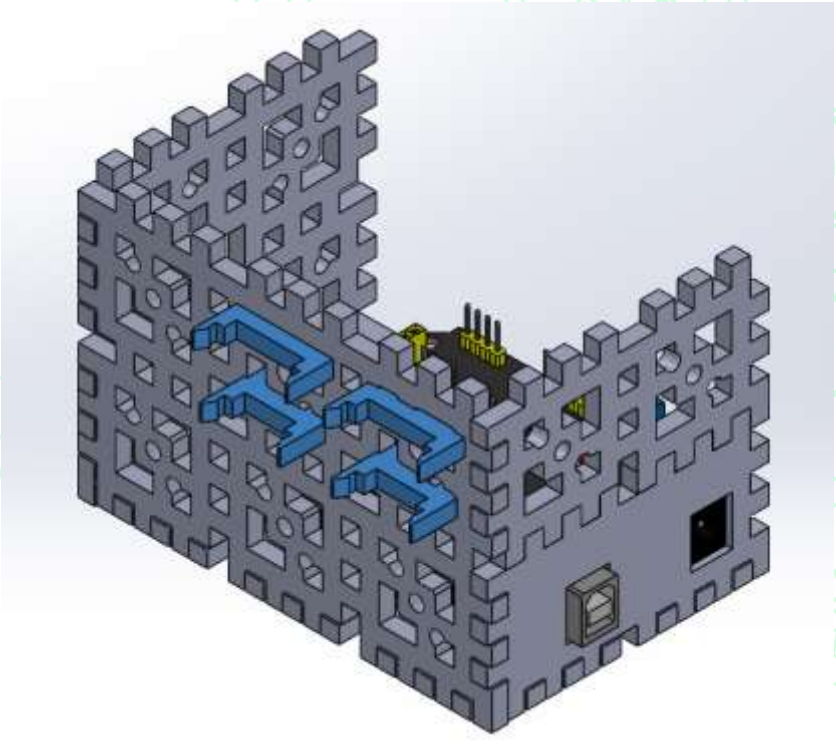
6. Додайте пластину 2x2.



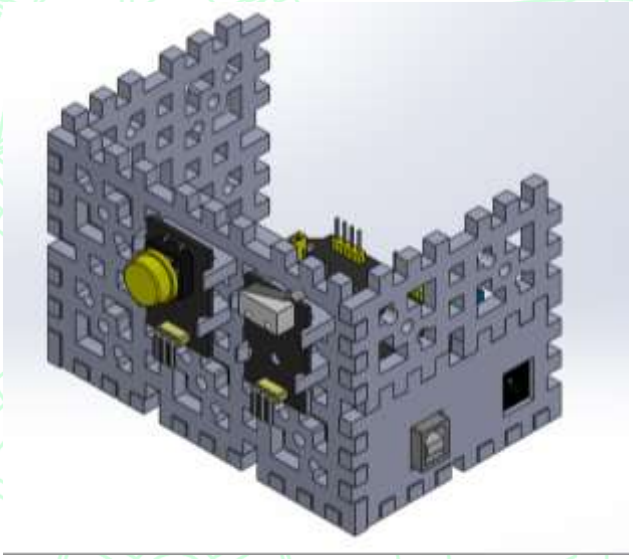
7. Додайте пластину 1x2 та зафіксуйте як показано на рисунку.



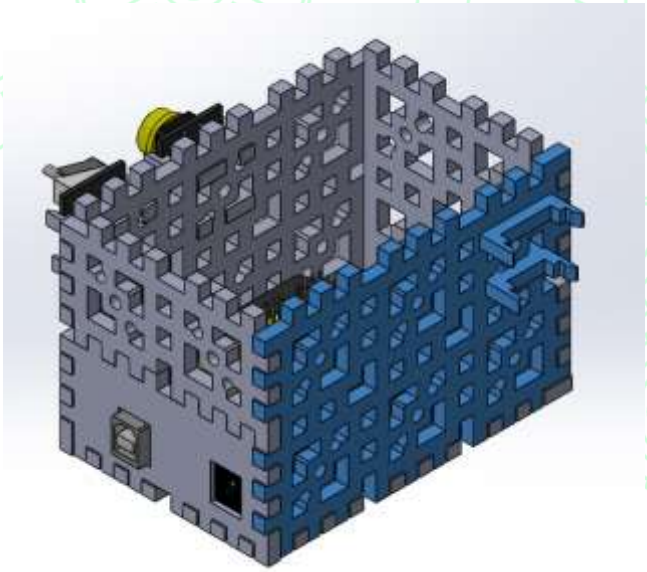
8. Додайте кріплення для фіксації модулів кінцевої кнопки та тактової кнопки як показано на рисунку.
Примітка. Ширший фіксатор зверху, вужчий – знизу.



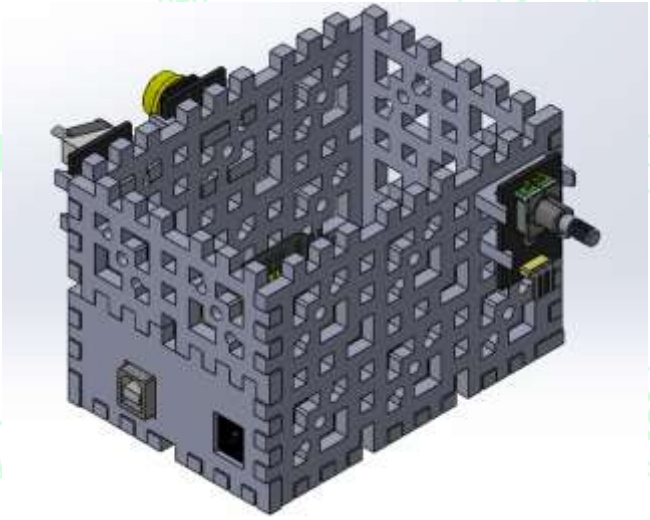
9. Зафіксуйте модуль тактової кнопки, та модуль кінцевої кнопки як показано на рисунку. Та протягніть дроти всередину через отвори в конструктивних елементах.



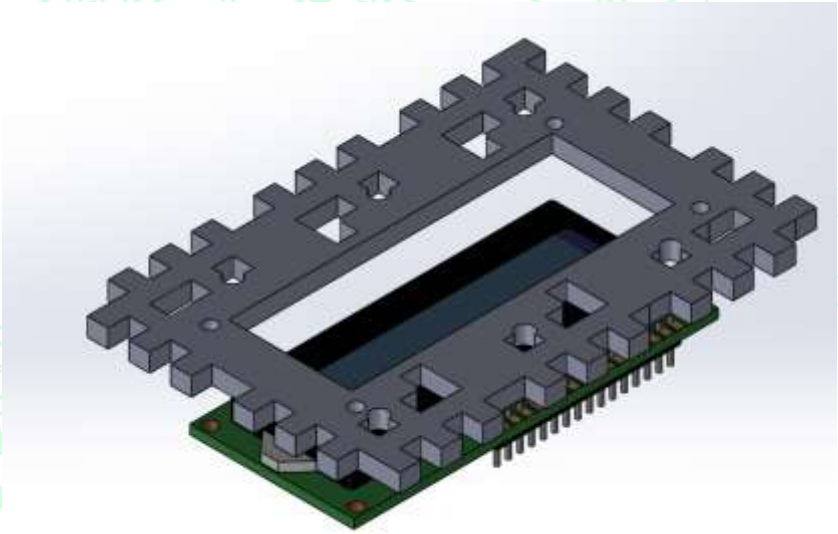
10. Додайте пластину 2x3 та кріплення для датчиків.



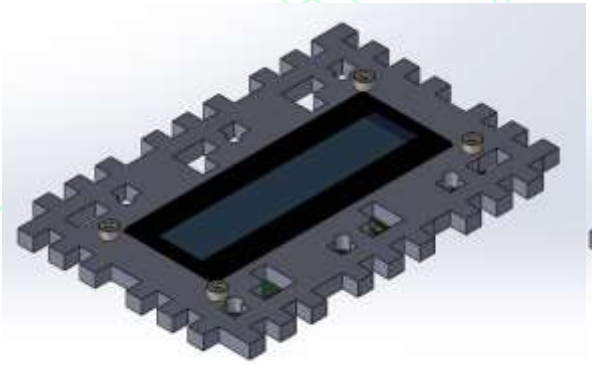
11. Зафіксуйте потенціометр як показано на рисунку.
Примітка. Під час підключення електронних компонентів, ви можете дещо послабити кріплення пластини для зручності роботи.

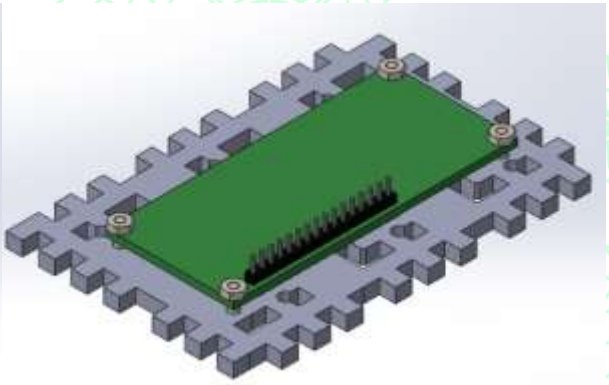


12. Зафіксуємо LCD дисплей на пластині. Для цього нам знадобиться пластина для кріплення LCD дисплею, сам дисплей, чотири болти M2.5 x10мм., та чотири гайки M2.5.
    Вставте LCD дисплей в пластину як показано на малюнку.



13. Зафіксуйте LCD дисплей за допомогою чотирьох болтів та гайок.





14. Наступним кроком необхідно під’єднати всі електронні компоненти відповідно до схеми. Спершу під’єднайте блютуз модуль, кінцеву та тактову кнопки до плати розширення.

Примітка. Для зручності сприйняття загальна схема розділена на дві, але по закінченню у вас повинні бути підключені всі компоненти.

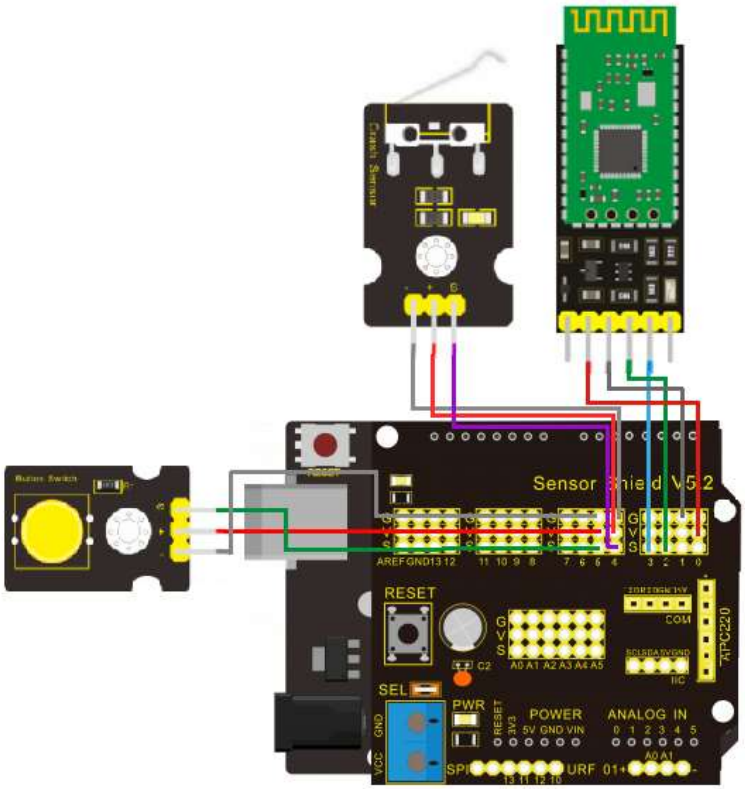


Схема підключення кінцевої кнопки, блютуз модуля, та тактової кнопки

15. Також під’єднайте LCD дисплей та потенціометр для регулювання контрастності до плати розширення як показано на схемі.

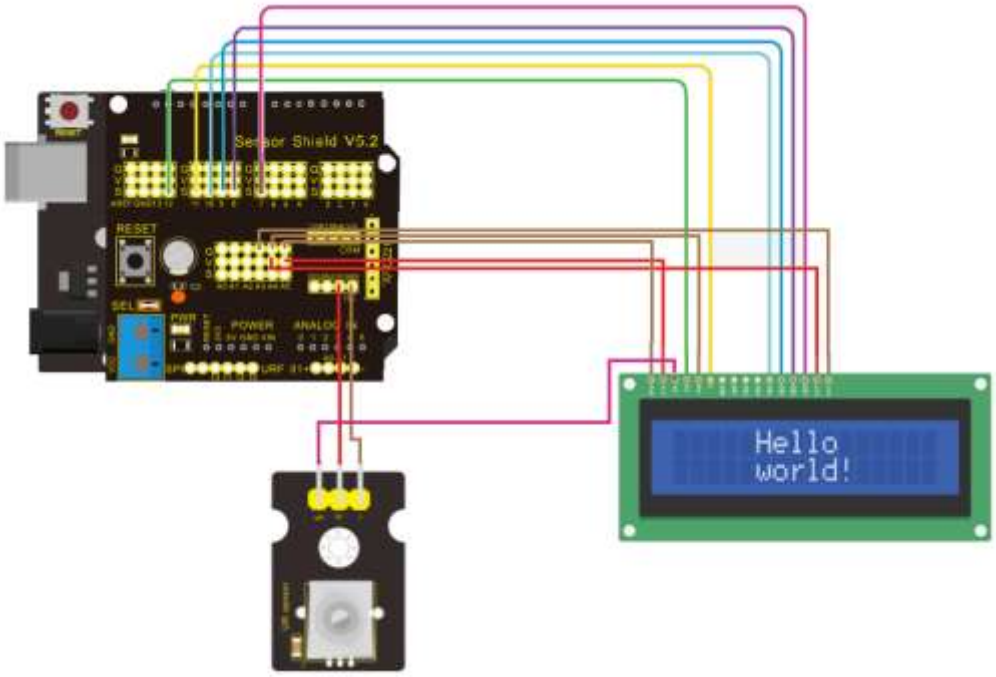
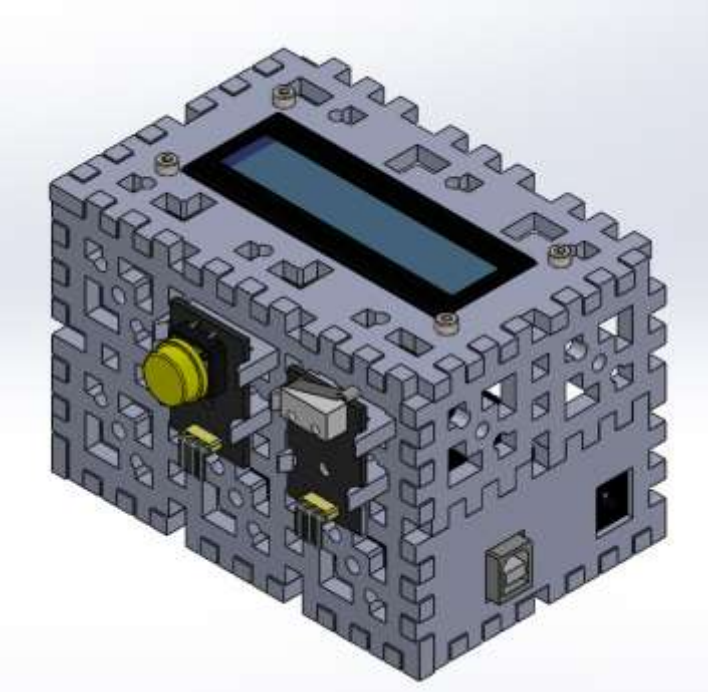


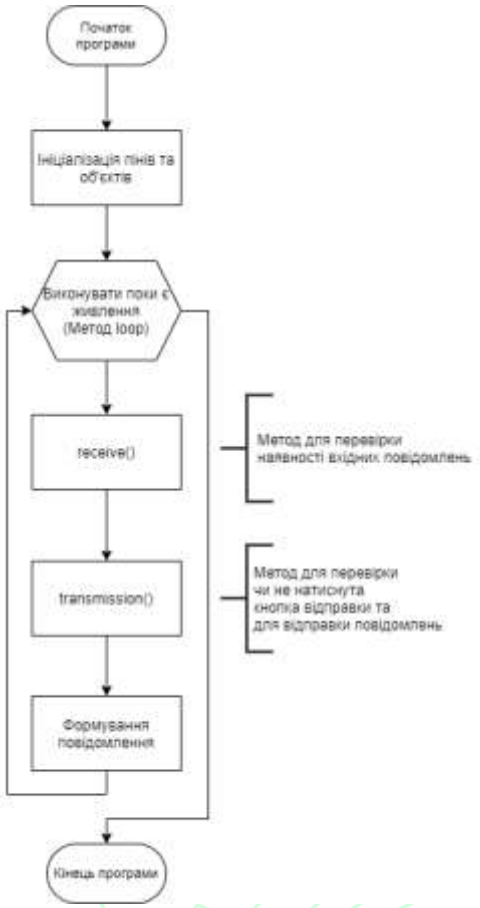
Схема підключення LCD дисплею до плати розширення

16. Зафіксуйте LCD дисплей на моделі, перед фіксацією уважно перевірте необхідні електронні підключення.



17. Після збирання моделі необхідно запрограмувати наш передавач Морзе.

Отже суть роботи в тому щоб зчитувати послідовність точок та тире за допомогою кінцевої кнопки, та передавати на іншу станцію за допомогою протоколу Bluetooth. Загальний принцип роботи показано на блок-схемі.



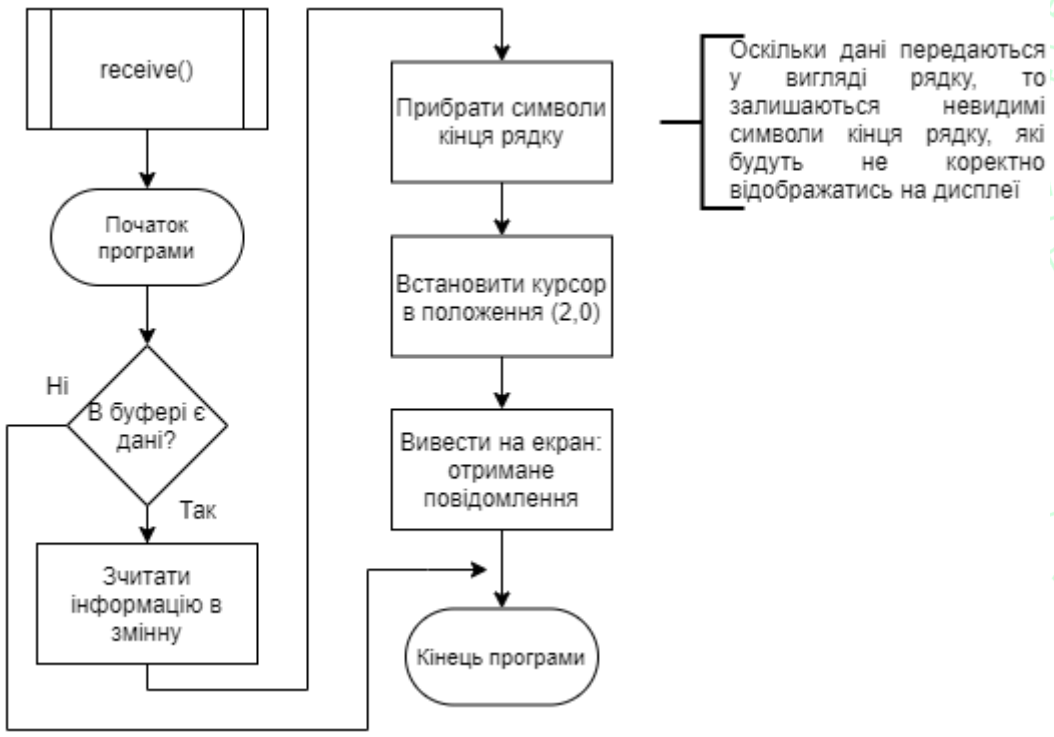
Перед тим як перейти до розробки програмного коду, гарним тоном буде визначити основні вимоги до нашого пристрою. Такий підхід дозволить краще зрозуміти поставлену задачу, і замість однієї великої і складної задачі, у вас вийде декілька менших задач, як за об’ємом так і за рівнем складності.

Отже, наш передавач Морзе, повинен:

1. Приймати повідомлення та відображати їх на дисплеї;
2. Формувати повідомлення з можливістю видалити неправильну літеру, в залежності від кількості натискань та їх характеру (коротке та довге натискання);
3. Якимось чином сигналізувати про помилку в роботі користувача, наприклад коли він ввів послідовність якої не існує в алфавіті, або намагається відправити пусте повідомлення;
4. І звичайно, відправляти повідомлення.

Таким чином ми отримали чотири задачі, різного рівня складності, які можуть бути реалізовані окремо одна від одної, а потім об’єднані в одну програму.

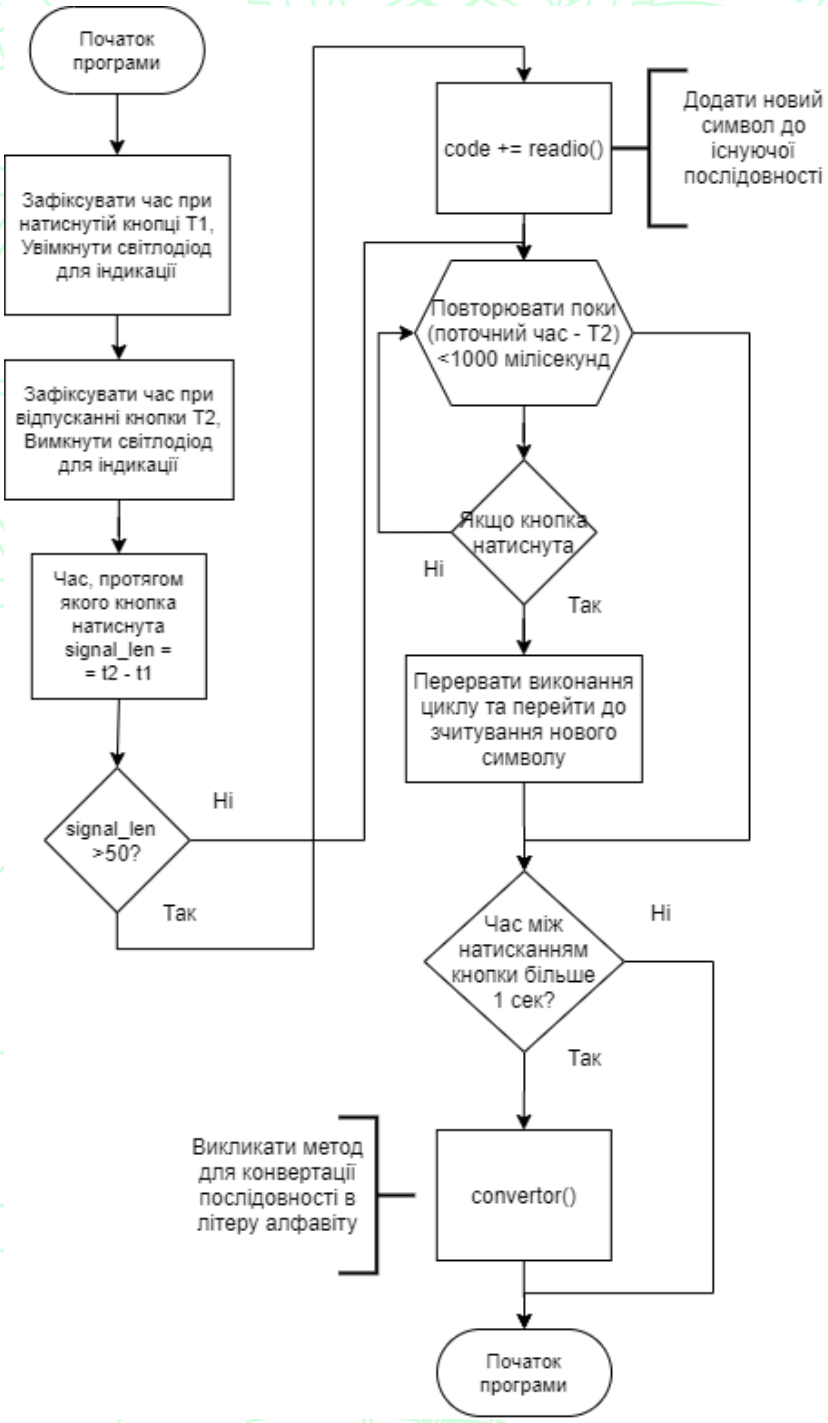
Розглянемо блок-схеми для реалізації кожної задачі, щоб краще зрозуміти принцип роботи всієї системи. Блок-схема методу для прийому повідомлень виглядає наступним чином:



Оскільки блютуз модуль працює як самостійний пристрій, який спілкується з мікроконтролером через послідовний інтерфейс, то коли він отримає повідомлення то помістить їх в буфер, де вони будуть очікувати поки їх прочитають. Відповідно нам необхідно постійно перевіряти чи не надійшла до буфера якась інформація, та якщо вона надійшла то просто зчитати та помістити її в змінну, після чого привести до «нормального» вигляду для відображення на дисплеї. Під нормальним виглядом мається на увазі чисте повідомлення без спеціальних символів закінчення рядку, які автоматично додаються в кінці повідомлення.

Після того як повідомлення було зчитано та відформатоване, виводимо його на екран. Для того щоб не було конфліктів між відображенням отриманого тексту та відображенням тексту що набирається, будемо виводити їх в різні рядки. Отже, перед виводом повідомлення на екран, необхідно встановити курсор в перший рядок, та відступити два символи зліва, щоб там додати підпис, що це за рядок. Після встановлення курсору просто виводимо повідомлення.

Перейдемо до другої задачі, яка є найскладнішою, отже нам необхідно створити програму яка зможе відрізняти коротке натискання від довгого, та знаходити відповідність між введеною послідовністю та літерою алфавіту, після чого додавати цю літеру в повідомлення. Розглянемо блок-схему:



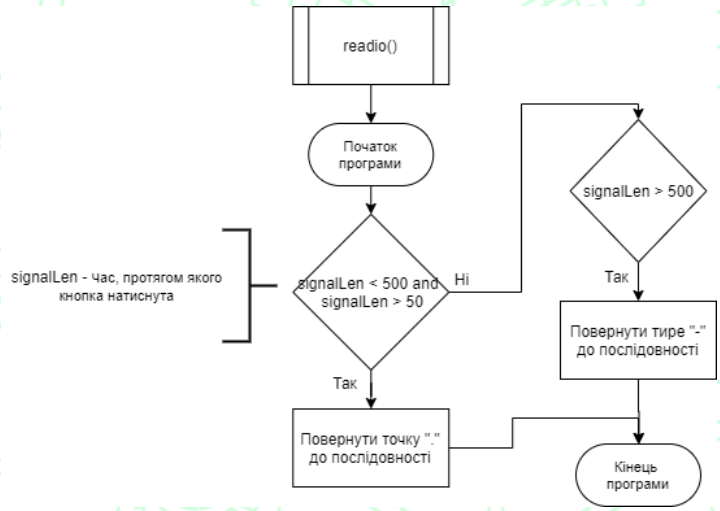
Логіка роботи досить проста, необхідно зафіксувати час після натискання та відпускання кнопки та знайти різницю, яка буде дорівнювати часу скільки кнопка була натиснута.

Нехай крапка буде дорівнювати часовому проміжку від 50 до 500 мілісекунд, а тире більше 500 мілісекунд. Отож, якщо кнопка була натиснута довше 50 мілісекунд ми викликаємо метод який у відповідності до часу поверне точку або тире до послідовності яка зберігається в змінній «**code**».

Після того як ми дізнались який символ був введений ми починаємо відлік часу до однієї секунди, якщо протягом секунди кнопка не була натиснута значить послідовність точок та тире введена і викликається метод для конвертації послідовності у відповідну літеру.

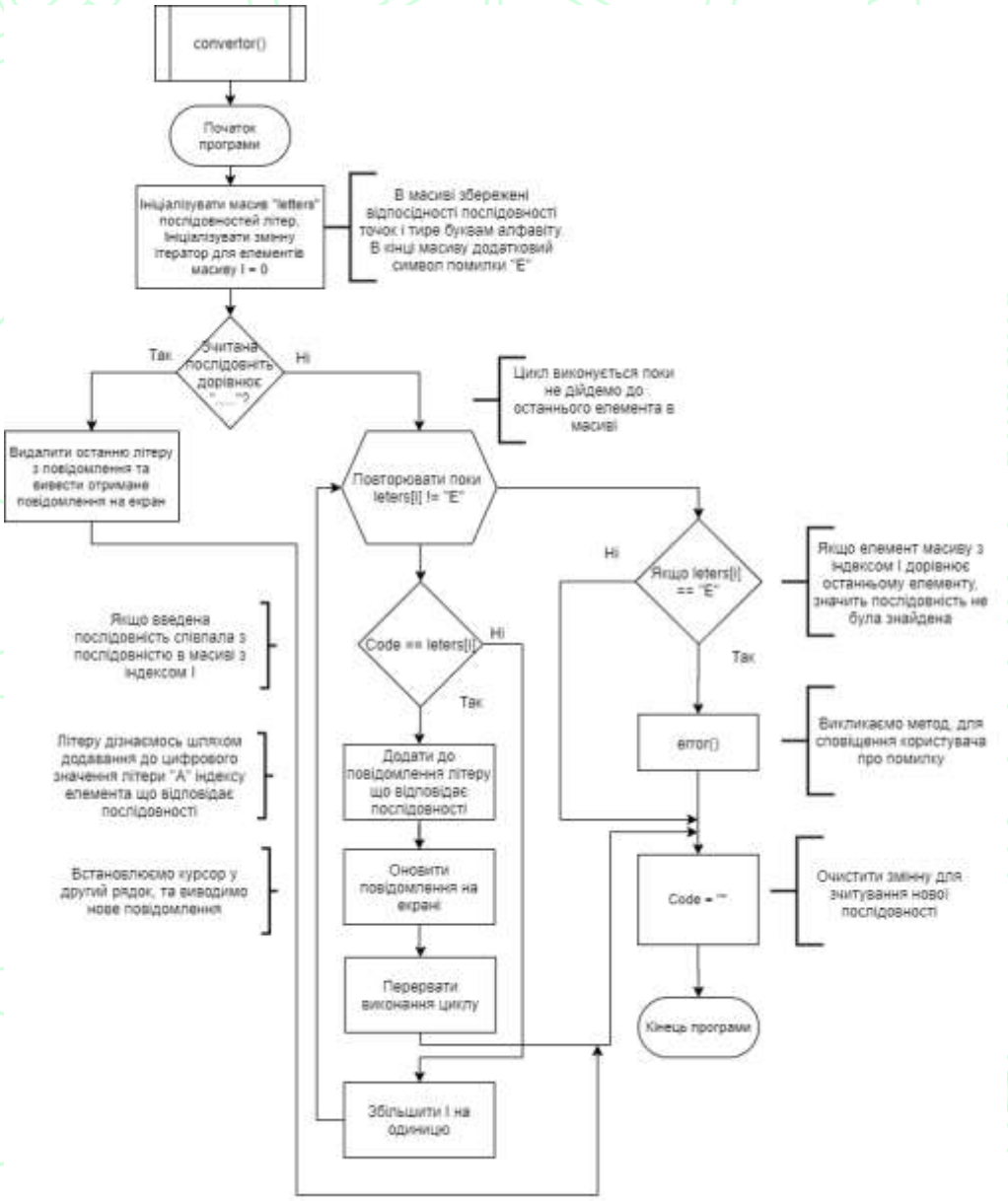
Якщо ж протягом секунди після останнього натискання кнопка знову була натиснута, переходимо до зчитування нового символу.

Блок-схема метода для визначення того який символ був введений, виглядає наступним чином:



Суть роботи методу дуже проста, необхідно перевірити скільки часу була натиснута кнопка. Якщо кнопка була натиснута від 50 до 500 мілісекунд тоді повертаємо точку до послідовності, якщо ж кнопка була натиснута більше 500 мілісекунд, значить повертаємо тире.

Після того як вся послідовність точок та тире була введена, викликається метод для конвертації послідовності в відповідну літеру або дію. Блок-схема методу наведена нижче:



В методі створюється масив всіх послідовностей точок та тире які дорівнюють літерам A…Z по порядку. В кінці масиву міститься додатковий елемент «E», він потрібен для того щоб визначити помилку, ми повернемось до нього дещо пізніше.

Отож після виклику метода та створення масиву, введена послідовність спершу порівнюється з послідовністю яка відповідає видаленню символу, нехай це буде п’ять точок. Для видалення останньої літери необхідно просто скопіювати початкове повідомлення без останнього символу, та записати в стару змінну, після чого вивести отримане повідомлення на екран. Після цього послідовність порівнюється з кожним елементом масиву по черзі в циклі. Якщо послідовність була знайдена, то індекс елементу в якому зберігається послідовність, додається до числового представлення літери «A» в системі кодування. Наприклад числове представлення літери A – 65, а послідовність дорівнює другому елементу масиву індекс якого дорівнює 1 (оскільки нумерація починається з нуля), таким чином ми отримуємо 65+1=66, що дорівнює літері “B” в системі кодування.

Якщо ми не знайшли в масиві жодної відповідності, змінна I буде вказувати на останній елемент масиву, тому що він перевірявся останнім. Перевіривши, що елемент дорівнює літері «E», ми можемо зробити висновок що послідовність була введена не правильно, та сповістити користувача про це.

В будь-якому разі, після конвертації нам необхідно очистити змінну «**code**» для наступної ітерації зчитування символів.

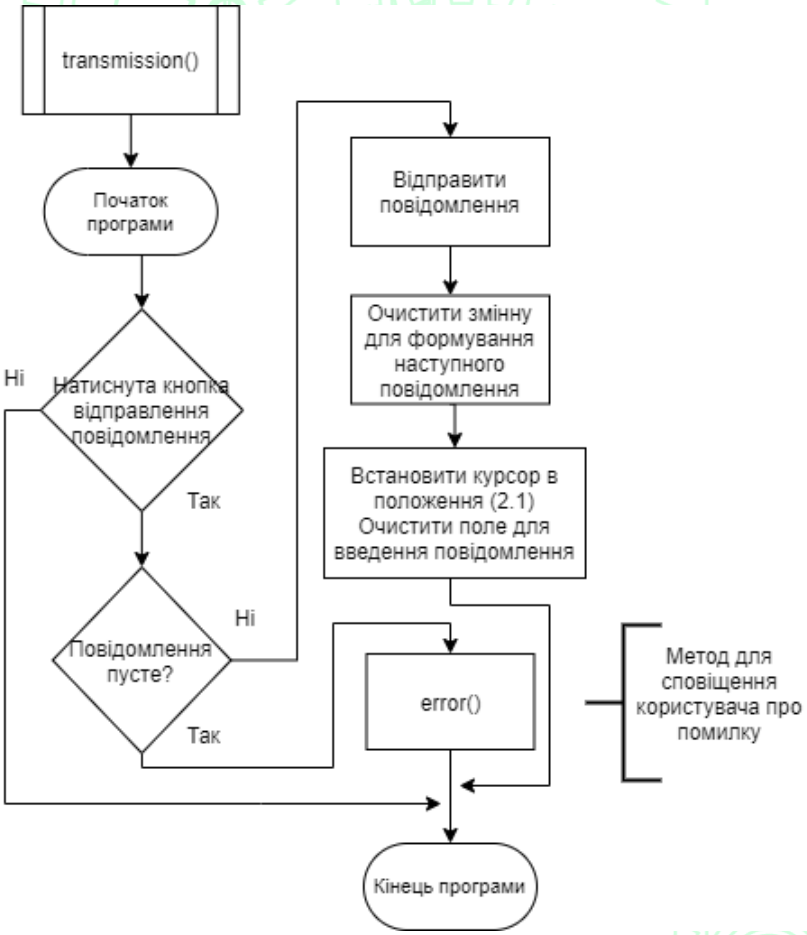
Примітка. В даному випадку літера «E» була взята як скорочення слова Error, але це не є обов’язковим.

Наступним кроком необхідно запрограмувати якийсь механізм індикації помилки, наприклад коли користувач ввів неіснуючу послідовність крапок та тире. Розглянемо блок-схему:



В даному випадку суть роботи методу дуже проста, ми просто на короткий проміжок часу в даному випадку 0.3 секунди, вимикаємо відображення всіх символів на дисплеї, після чого знову вмикаємо. Це дуже простий спосіб індикації який ніяким чином не вплине на зручність роботи, але дасть користувачу зрозуміти коли він зробив помилку.

Ну і остання задача, це можливість відправляти повідомлення, оскільки ми маємо вже сформоване повідомлення, зробити це досить легко. Давайте поглянемо на реалізацію метода відправки повідомлень через блок-схему.



Відправка повинна бути зав’язана на окрему кнопку, яку користувач натисне після вводу повідомлення. Отже, якщо користувач натиснув кнопку, необхідно перевірити чи є в повідомлення якийсь текст, адже кнопка могла бути натиснута помилково. Якщо тексту немає, то викличемо вже відомий нам метод сигналізації про помилку. Якщо повідомлення не пусте, то відправляємо його через послідовний порт, до якого підключено блютуз модуль. Після чого необхідно очистити змінну в яку вводився текст, для того щоб в неї можна було записати нове повідомлення. Та очистимо місце на дисплеї для вводу нового повідомлення.

18. Після того як ми проаналізували всі блок-схеми для рішення поставлених задач та розібрались з логікою роботи окремих складових нашої програми, можна перейти до реалізації блок-схем у вигляді програмного коду.

```
#include <LiquidCrystal.h>    //Бібліотека для роботи з дисплеєм 1602
#include "SoftwareSerial.h"   //Бібліотека для емуляції послідовного порту
#define bRX_Pin 2             //ініціалізація пінів для віртуального порту
#define bTX_Pin 3
unsigned long signal_len,t1,t2;   //змінні для фіксації скільки часу кнопка була натиснута
int inputPin = 4;                 //кнопка для введення повідомлення
int enterButton = 5;              //кнопка відправки
int ledPin = 13;                  //вбудований світлодіод
String code = "";                 //змінна для зберігання послідовності точок і тире
LiquidCrystal lcd(12, 11, 10, 9, 8, 7);   // (RS, E, DB4, DB5, DB6, DB7)
SoftwareSerial mySerial(bRX_Pin, bTX_Pin);//ініціалізація послідовного порту для Bluetooth модуля
String message = "";              //змінна для зберігання повідомлення для відправки
void setup() {
  Serial.begin(9600);
  mySerial.begin(9600);           //Швидкість віртуального порту
  pinMode(inputPin, INPUT);
  pinMode(ledPin,OUTPUT);
  pinMode(enterButton, INPUT);
  //ініціалізація дисплею та відображення символів для підпису
  //полів для отриманих та відправлених повідомлень
  lcd.begin(16, 2);
  lcd.print("r-");
  lcd.setCursor(0,1);
  lcd.print("t-");
}

void loop()
{
  while(digitalRead(inputPin) == HIGH) {    //цикл який працює до тих пір поки кнопка не натиснута
   receive();                               //метод для прийому повідомлень
   transmission();                          //метод для відправки повідомлень
  }
  t1 = millis();                            //час коли кнопку натиснули
  digitalWrite(ledPin, HIGH);
  while (digitalRead(inputPin) == LOW) {}   //цикл який працює до тих пір поки кнопку не відпустили
  t2 = millis();                            //час коли кнопку відпустили
  digitalWrite(ledPin, LOW);
  signal_len = t2 - t1;                     //різниця часу між натисканням та відпусканням кнопки
  if (signal_len > 50)                      //якщо кнопка була натиснута більше 50 мілісекунд
  {
    code += readio();                       //викликаємо метод для визначення точки або тире
  }
  while ((millis() - t2) < 1000)            //цикл який працює 1 секунду після останнього натискання
  {
    if (digitalRead(inputPin) == LOW)       //якщо кнопку протягом цього часу натиснули
    {
      break;                                //перервати виконання циклу
    }
  }
  if((millis() - t2) >= 1000){              //якщо кнопку не натискали більше однієї секунди
    convertor();                            //викликаємо метод для конвертації послідовності точок і тире
  }                                         //в літеру алфавіту
}
```

```
char readio()                                   //метод для визначення точок та тире
{
  if (signal_len < 500 && signal_len > 50)      //якщо кнопка була натиснута менше 500 мілісекунд
  {
    return '.';                                 //метод повертає точку до послідовності
  }
  else if (signal_len > 500)                    //якщо була натиснута більше ніж 500 мілісекунд
  {
    return '-';                                 //метод повертає тире до послідовності
  }
}
```

```
void convertor()
{
  //масив послідовностей точок і тире що відповідають літерам від A до Z,
  //та додатковий елемент "E" що знаходиться за літерою Z, для визначення помилки введення
  static String letters[] = {".-", "-...", "-.-.", "-..", ".", "..-.", "--.", "....", "..",
                             ".---", "-.-", ".-..", "--", "-.", "---", ".--.", "--.-",
                             ".-.", "...", "-", "..-", "...-", ".--", "-..-", "-.--", "--..", "E"
                            };
  int i = 0;
  if (code == ".....")                          //якщо послідовність дорівнює п'яти точкам
  {
    //копіюємо текст зі старого повідомлення, без останнього символу
    //тобто видаляємо останній введений символ
    message = message.substring(0, message.length()-1);
    lcd.setCursor(2,1);                         //встановлюємо курсор в другий рядок
    lcd.print(message + "             ");      //виводимо нове повідомлення на дисплей,
  }                                             //пусті символи використовуються в якості очистки дисплею
  else
  {
    while (letters[i] != "E")                   //цикл який працює поки не зустріне елемент масиву "E"
    {
      if (letters[i] == code)                   //якщо введена послідовність співпадає з послідовністю в масиві
      { //додаємо в повідомлення літеру що відповідає послідовності
        //шляхом додавання до числового представлення літери "A" індексу елемента якому відповідає послідовність
        message += char('A' + i);
        lcd.setCursor(2,1);                     //встановлюємо курсор у другий рядок
        lcd.print(message + "             "); //виводимо нове повідомлення
        break;                                  //виходимо з циклу
      }
      i++;
    }
    if (letters[i] == "E")                      //якщо дойшли до останнього елемента
    {
       error();                                 //сигналізуємо про помилку, оскільки введеній послідовності
    }                                           //не відповідає жоден символ
  }
  code = "";                                    //очищаємо змінну для нової послідовності
}
```

```
void error(){                                   //метод для індикації помилки
  lcd.noDisplay();                              //вимикаємо відображення символів
  delay(300);
  lcd.display();                                //вмикаємо відображення символів
}
```

```
void receive(){                                  //метод для прийому повідомлень
  if(mySerial.available()){                      //якщо в буфері є непрочитані дані
    String mReceive = mySerial.readString() ;    //зчитуємо дані у вигляді рядку
    mReceive.trim();                             //видаляємо символ кінця рядку із зчитаного рядка
    lcd.setCursor(2, 0);                         //встановлюємо курсор в перший рядок
    lcd.print(mReceive+"              ");        //виводимо отримане повідомлення на дисплей
  }
}
void transmission(){                             //метод для передачі повідомлення
  if(digitalRead(enterButton) == 0){             //якщо кнопка відправлення натиснута
    if(message == ""){                           //якщо повідомлення порожне
      error();
      return;                                    //перериваємо виконання методу
    }else{
      mySerial.println(message);                 //відправляємо повідомлення
      message = "";                              //очищаємо змінну для формування нового повідомлення
      lcd.setCursor(2,1);                        //встановлюємо курсор на другий рядок
      lcd.print(message + "              ");     //виводимо пусте повідомлення,
    }                                            //тим само очищаючи місце для введення нового повідомлення
  }
}
```

19. Завантажте програмний код до мікроконтролера. Для тестування пристрою об’єднайтесь в групи, щоб в кожній групі було два пристрої, та підключіть пристрої один до одного за допомогою Bluetooth модулів. За необхідністю скористайтесь підказками наведеними в другому завданні лабораторної роботи №8.

    Примітка. За бажанням ви також можете підключити пристрій до телефону як в першому завданні лабораторної роботи №8.
20. Протестуйте всі можливості пристрою, а саме:
    a. Введення послідовності точок тире, шляхом довгих та коротких натискань кінцевої кнопки, після введення необхідної послідовності у другому рядку повинна з’явитись відповідна літера;
    b. Наявність індикації помилок при неправильній послідовності, або спробі відправити пусте повідомлення;
    c. Можливість видалення останнього символу, шляхом введення послідовності з п’яти точок.
    d. Можливість відправки повідомлень, шляхом натискання тактової кнопки.
    e. Можливість отримання повідомлень, отримане повідомлення повинно відображатись у верхньому рядку.

Самостійна робота. Працюючи в командах знайдіть за допомогою мережі інтернет, інформацію як створювати свої власні символи для виводу на LCD дисплей. Створіть 5 власних символів для відображення на дисплеї, та придумайте власні послідовності точок і тире для їх кодування. Реалізуйте можливість відправлення та отримання таких символів.